Object-Oriented Design and Extension Report

Architecture and responsibilities

The application is a local, turn-based artillery game implemented with Java and Processing. App extends PApplet and owns the main game loop, configuration loading, terrain, round transitions, player inputs and HUD. The nested Tanks class stores each player's health, fuel, parachutes, score, turret angle and shield state. It implements movement, aiming, power adjustment and firing. Projectiles represents an individual shot, storing its firing tank, precise position, velocity and wind at launch. Tree stores a tree's position, while DeathExplosion stores the lifetime, position and owner of the explosion triggered by a destroyed tank.

Object relationships and state

App maintains collections of tanks, projectiles, trees and active death explosions. Each projectile references the tank that fired it. Each death explosion identifies its victim and, where available, the tank responsible. This ownership is essential for awarding only effective enemy damage. A per-player score map persists across levels while tank health, fuel, parachutes and shields reset for each level. The class diagram reflects the actual code: entities are non-static inner classes of App, not separate top-level Java classes.

Gameplay algorithms and extension

The simulation runs at 30 frames per second. Tanks move using remaining fuel; turret angles and power are updated continuously while the relevant key is held. Projectile coordinates use floating-point precision. Vertical velocity is affected by gravity, and horizontal velocity by the wind recorded at launch. Terrain is derived from character layouts, expanded into pixel columns and smoothed twice using a 32-sample moving average. Explosions damage nearby tanks according to their distance and deform the terrain. Unsupported tanks descend with a parachute when available or lose health without one.

The custom Shield extension is purchased with the H key for 20 score points. A player cannot buy it without sufficient score or while already shielded. A shield absorbs the next damaging blast without reducing health or awarding points to the attacker. It then disappears. A transparent blue bubble and the Shield ON HUD label communicate its state. Shields intentionally do not protect against fall damage, and they do not persist across levels.

Testing and design trade-offs

JUnit covers physics, damage, turn management, power-ups, results, edge cases and headless rendering calls. The last supplied Windows verification reported 147 tests passed and 93.20% average JaCoCo instruction/branch coverage. This does not establish pixel-perfect rendering or prove every game scenario.

The current design has a clear limitation: App manages several responsibilities, and its mutable static collections create coupling between entities and tests. A future refactor could extract GameState, PhysicsEngine, Terrain, Renderer and PowerUp classes behind interfaces while retaining the existing test suite as regression protection.